C# SKILLS SHARPENER • MILESTONE 3 MACHINE PROBLEM

Paws & Whiskers: Pet Sanctuary & Veterinary Operations Terminal

Focus: Advanced Control Flow, Decision Logic, Loops, Scopes & Defensive String Operations

1. Problem Scenario

You are contracted as a software engineer by **Paws & Whiskers Sanctuary & Veterinary Center**, an animal rescue and specialized veterinary adoption agency. The sanctuary coordinates animal intake, triage health assessments, specialized dietary assignments, boarding billing, adoption matchmaking, and batch manifest archiving.

The sanctuary is retiring its vulnerable manual spreadsheet system and requires a high-reliability, console-driven management terminal. The system must process inbound pet registration barcode tags, extract medical metadata without regular expressions, screen animals for contagious health risks using multi-branch decision logic, calculate adoption subsidies and boarding costs using switch-case and ternary operators, and update in-memory medical records in-place using indexed `for` loops.

Your task is to build a robust, modular, multi-file C# console application that proves mastery over **Milestone 3 core competencies** while maintaining zero-crash defensive coding standards:

- **Defensive User Ingestion:** Safe console handling with nullable strings (`string?`), null-coalescing (`?? ""`), and non-crashing `int.TryParse` and `decimal.TryParse` range loops.
- **String Parsing via Delimiter Extraction:** Slicing raw barcode health lines with `IndexOf`, `Substring`, `Remove`, and `TrimStart` instead of high-level regular expressions.
- **Boolean Logic & Normalization:** Normalizing strings with `.Trim().ToLower()`, evaluating substrings with `.Contains()`, and applying logical inversion with `!`.
- **Conditional / Ternary Operator (`?:`):** Computing VIP adoption discounts, medical rehabilitation subsidies, and inline formatted messages.
- **Multi-Label `switch-case`:** Grouping species classifications with stacked case labels and fall-through prevention via `break`.
- **`for` vs `foreach` Array Mutation:** Understanding the read-only constraint of `foreach` and using indexed `for` loops to mutate records in-place.
- **Reverse Sequence Traversal:** Traversing audit ledgers in reverse-chronological order (newest intake first) using countdown loops (`for (int i = count - 1; i >= 0; i--)`).
- **Loop Control:** Driving interactive menus via `do-while`, skipping corrupted batch rows with `continue`, and terminating sessions cleanly with `break`.

2. Learning Competencies Assessed

Competency	Concrete Application in Paws & Whiskers Terminal
Defensive Ingestion	Guarding all terminal prompts with `while` loops; verifying numeric bounds via `int.TryParse` and `decimal.TryParse`; eliminating runtime format crashes.
String Delimiter Parsing	Extracting key-value pairs from delimited pet health tags using `IndexOf(":")`, `Substring()`, `Remove()`, and `TrimStart()` without `.Split()`.
Boolean Normalization	Sanitizing user commands and species names via `.Trim().ToLower()`; testing keywords with `.Contains()`; inverting conditions with `!`.
Conditional Operator (`?:`)	Applying inline ternary expressions for VIP adopter discounts, high-risk quarantine surcharges, and inline string interpolation.
Branching via `switch-case`	Evaluating animal species with stacked labels (e.g. `case "dog": case "puppy":`); applying default fallback logic; avoiding missing `break` errors.
In-Place Collection Mutation	Using indexed `for` loops to modify parallel array/list items in-place (overcoming the C# `CS1656` read-only limitation of `foreach`).
Reverse Traversal	Auditing intake ledgers in reverse order using zero-based countdown indexing (`int i = count - 1; i >= 0; i--`).
Continuous Execution	Orchestrating an interactive dispatch menu inside a `do-while` loop; bypassing corrupted records via `continue`; exiting via `break`.

3. Segregated Namespace Architecture

You must separate computation algorithms from user interaction across two segregated namespaces:

- `PetCare.Core`: Contains `PetEngine.cs` (pure calculation algorithms, string extraction, health risk screening, and in-place array mutation). No direct file I/O or console printing.
- `PetCare.IO`: Contains `PetIO.cs` (batch CSV file readers, manifest exporters, interactive console input guards, and tabular reporting).

C# SKILLS SHARPENER • MILESTONE 3 MACHINE PROBLEM

Required Modular Methods in PetCare.Core.PetEngine

Namespace: PetCare.Core • Class: PetEngine

Decompose all business logic into testable, pure static methods returning typed C# tuples or data structures:

Method A: Low-Level Barcode Tag Delimiter Extractor

```
public static (string tokenKey, string tokenValue, string remainingTag) ExtractTagToken(string rawTagLine)
```

Logic: 1. Finds the first colon delimiter `:` using `rawTagLine.IndexOf(":")`.
2. If missing (`-1`), returns `("UNKNOWN", rawTagLine.Trim(), "")`.
3. Uses `Substring(0, colonPos)` to extract `tokenKey`, normalized with `.Trim().ToUpper()`.
4. Slices remainder using `rawTagLine.Substring(colonPos + 1).TrimStart()`.
5. If a semicolon `;` exists in remainder (`IndexOf(";")`), extracts `tokenValue` via `remainder.Substring(0, semiPos).Trim()`, and `remainingTag` via `remainder.Remove(0, semiPos + 1).TrimStart()`.
6. If no semicolon exists, entire remainder is `tokenValue`, and `remainingTag` evaluates to empty string `""`.

Method B: Health Risk & Dietary Screening Evaluator

```
public static (bool needsQuarantine, string dietPlan, int healthRiskScore) ScreenHealthStatus(string species, int ageYears, bool isVaccinated, bool hasSpecialNeeds, string medicalHistoryNotes)
```

Logic:
• Normalizes notes: `string notes = medicalHistoryNotes.Trim().ToLower();`
• **Quarantine Condition (Risk Score = 90):** If `!isVaccinated` AND (`notes.Contains("fever")` OR `notes.Contains("contagious")` OR `notes.Contains("parvo")` OR `notes.Contains("respiratory")`), set `needsQuarantine = true` and `healthRiskScore = 90`.
• **Observation Condition (Risk Score = 55):** Else if `hasSpecialNeeds` OR `ageYears >= 10` OR `!isVaccinated`, set `needsQuarantine = false` and `healthRiskScore = 55`.
• **Normal Condition (Risk Score = 10):** Otherwise, `needsQuarantine = false` and `healthRiskScore = 10`.
• **Dietary Assignment:** If `ageYears <= 1`, return `"High-Protein Growth Formula"`; else if `ageYears >= 8`, return `"Senior Mobility & Renal Care"`; otherwise return `"Standard Balanced Organic Diet"`.

Method C: Adoption Fee & Boarding Cost Calculator

```
public static (decimal baseFee, decimal boardingTotal, decimal medicalSubsidy, decimal finalAdoptionTotal) CalculateAdoptionCosts(string species, int daysInSanctuary, bool isAdopterVip, int healthRiskScore)
```

Logic:
• **Species Base Fee & Daily Rate via `switch-case`:**
- `case "dog": case "puppy":` Base Fee = ₱3,500.00 | Boarding = ₱250.00/day
- `case "cat": case "kitten":` Base Fee = ₱2,200.00 | Boarding = ₱180.00/day
- `case "rabbit": case "hamster":` Base Fee = ₱1,200.00 | Boarding = ₱100.00/day
- `case "bird": case "parrot":` Base Fee = ₱1,800.00 | Boarding = ₱120.00/day
- `default:` Base Fee = ₱1,500.00 | Boarding = ₱150.00/day
• **Boarding Total:** `boardingTotal = daysInSanctuary * dailyRate;`
• **Ternary Medical Subsidy:** If `healthRiskScore >= 50`, calculate subsidy: `healthRiskScore >= 80 ? 1200.00m : 600.00m;` otherwise ₱0.00.
• **Ternary VIP Adopter Discount:** `decimal discountRate = isAdopterVip ? 0.20m : 0.05m;`
• **Final Total:** `(baseFee + boardingTotal - medicalSubsidy) * (1.0m - discountRate)`. Guard: minimum ₱500.00.

Method D: In-Place Status Mutation Engine (Indexed For-Loop)

```
public static int UpdateAdoptionLedger(string[] petIds, string[] adoptionStatuses, bool[] quarantineFlags, string[] targetIdsToAdopt)
```

Logic: Must iterate through arrays using a counted `for (int i = 0; i < petIds.Length; i++)` loop. For each animal whose ID matches one in `targetIdsToAdopt`:
• If `quarantineFlags[i] == true`, do NOT adopt; mutate `adoptionStatuses[i] = "HELD_QUARANTINE"`.
• Else if `adoptionStatuses[i] == "AVAILABLE"`, mutate `adoptionStatuses[i] = "ADOPTED"` and increment tally.
• *Crucial Pedagogical Takeaway:* Demonstrates why C# read-only `foreach` loops cannot mutate array cells, whereas indexed `for` loops directly modify memory. Returns total count of newly adopted pets.

Method E: Sanctuary Batch Analytics Generator

```
public static (int totalPets, int availableCount, int adoptedCount, int quarantineCount, decimal totalRevenue, decimal avgDaysInCare) GenerateSanctuarySummary(string[] statuses, bool[] quarantineFlags, decimal[] fees, int[] daysInCare)
```

Logic: Iterates across parallel records. Tallies counts per category, sums total collected revenue, and calculates average days in care guarded with `if (totalPets > 0)` to prevent `DivideByZeroException` on empty batches.

C# SKILLS SHARPENER • MILESTONE 3 MACHINE PROBLEM

Interactive Console Menu & Operational Flow

Namespace: PetCare.IO • Interactive Terminal Specification

4. Operational Flow & Dispatch Terminal

The program entry point must execute within a continuous `do-while` loop displaying the primary dispatch terminal:

```
============================================================
    PAWS & WHISKERS: PET SANCTUARY & ADOPTION TERMINAL (M3)
============================================================
  1. Intake Single Pet (Interactive Delimiter Slicing)
  2. Ingest Batch Pet Inventory from File (pet_inventory_batch.txt)
  3. Execute In-Place Adoption Mutation (Indexed For-Loop)
  4. View Sanctuary Registry & Reverse Audit Ledger
  5. Export Audited Adoption Manifest (pet_adoption_manifest.txt)
  6. Exit Terminal Session
============================================================
Enter option (1-6):
```

5. Menu Option Specifications

- **Option 1 (Intake Single Pet):** Interactively prompts the operator for raw barcode tag string (e.g. `TAG: P-105; SPECIES: Cat; AGE: 2; VAX: true; NEEDS: false; DAYS: 5; NOTES: Healthy rescue`). Calls `ExtractTagToken` iteratively inside a `while` loop to parse fields, computes screening and fees, and appends to in-memory parallel collections.
- **Option 2 (Ingest Batch Pet Inventory from File):** Reads `pet_inventory_batch.txt` from disk using `StreamReader` or `File.ReadAllLines` enclosed in a `using` statement. Validates row token count (`parts.Length >= 9`) and uses `continue` to cleanly skip corrupted rows. Parses numeric/boolean fields defensively via `TryParse`.
- **Option 3 (Execute In-Place Adoption Mutation):** Prompts for one or more target Pet IDs to adopt. Invokes `UpdateAdoptionLedger` to mutate the status of matched records in-place using a counted `for` loop. Reports the count of updated records.
- **Option 4 (View Sanctuary Registry & Reverse Audit Ledger):** Prompts operator for display ordering:

  > `Select Display Order: [1] Forward (Intake Order) | [2] Reverse (Newest First):`

  If 2 is selected, executes reverse array traversal using a countdown loop: `for (int i = count - 1; i >= 0; i--)`. Prints the aligned tabular registry followed by batch summary metrics.
- **Option 5 (Export Audited Adoption Manifest):** Formats all active records and summary analytics into a pipe-delimited report written to `pet_adoption_manifest.txt` using a `StreamWriter`.
- **Option 6 (Exit Terminal Session):** Prints a formal closing message and terminates the session cleanly using `break`.
- **Defensive Input Validation:** If the operator enters an invalid menu option or malformed value, display a friendly error message and reprompt without crashing.

6. Console Display Specifications (Option 4 Sample)

When Option 4 is triggered, output a formatted, aligned table with computed metrics:

```
=================================================================================================================
                        PAWS & WHISKERS: ACTIVE SANCTUARY REGISTRY
=================================================================================================================
Pet ID    Name         Species    Age  Vax?  Risk  Days  Diet Plan                 Status     Total Fee
-----------------------------------------------------------------------------------------------------------------
P-101     Bella        Dog          3  Yes     10    12  Standard Balanced Organic  AVAILABLE  ₱6,175.00
P-102     Milo         Cat          1  No      90    20  High-Protein Growth        QUARANTINE ₱3,990.00
P-103     Barnaby      Rabbit       4  Yes     10     5  Standard Balanced Organic  ADOPTED    ₱1,615.00
P-104     Jasper       Dog         10  Yes     55    45  Senior Mobility & Renal    AVAILABLE  ₱13,442.50
-----------------------------------------------------------------------------------------------------------------
SANCTUARY REGISTRY ANALYTICS:
  Total Animals Ingested : 4 pets
  Available for Adoption : 2 pets
  Adopted / Placed       : 1 pet
  Quarantine Hold        : 1 pet
  Total Adoption Revenue : ₱25,222.50
  Average Days in Care   : 20.5 days
=================================================================================================================
```

C# SKILLS SHARPENER • MILESTONE 3 MACHINE PROBLEM

File Archival, Test Scenarios & Verification Checklist

Data Formats • Test Matrix • Quality Control Standards

7. File Storage & Archival Specifications

1. Input Batch File (`pet_inventory_batch.txt`): Standard comma-separated structure:

```
PetId,Name,Species,Age,Vaccinated,SpecialNeeds,DaysInCare,HistoryNotes,Status
P-101,Bella,Dog,3,true,false,12,Regular intake,AVAILABLE
P-102,Milo,Cat,1,false,false,20,Fever and respiratory cough,AVAILABLE
P-103,Barnaby,Rabbit,4,true,false,5,Surrendered healthy,AVAILABLE
P-104,Jasper,Dog,10,true,true,45,Arthritis senior rescue,AVAILABLE
```

2. Export Manifest File (`pet_adoption_manifest.txt`): Pipe-delimited output generated by Option 5:

```
AUDITED SANCTUARY ADOPTION & BOARDING MANIFEST
Generated: 2026-10-25
--------------------------------------------------------------------------------
P-101 | Bella | Dog | Age 3 | Vax: True | Status: AVAILABLE | Fee: ₱6,175.00
P-102 | Milo | Cat | Age 1 | Vax: False | Status: QUARANTINE | Fee: ₱3,990.00
P-103 | Barnaby | Rabbit | Age 4 | Vax: True | Status: ADOPTED | Fee: ₱1,615.00
P-104 | Jasper | Dog | Age 10 | Vax: True | Status: AVAILABLE | Fee: ₱13,442.50
--------------------------------------------------------------------------------
SUMMARY: Total=4 | Available=2 | Adopted=1 | Quarantine=1 | Revenue=₱25,222.50
```

8. Verification & Test Scenarios

Pet ID	Input Attributes	Calculated Financials & Diet	Assigned Status
P-101	Bella • Dog • Age 3 Vax: Yes • SpecialNeeds: No Days: 12 • VIP: No (5% off)	Base: ₱3,500 | Boarding (12×₱250): ₱3,000 Subsidy: ₱0.00 | Net: ₱6,500 × 0.95 = **₱6,175.00** Diet: Standard Balanced Organic	AVAILABLE
P-102	Milo • Cat • Age 1 Vax: No • Notes: Fever & cough Days: 20 • VIP: No	Base: ₱2,200 | Boarding (20×₱180): ₱3,600 Risk: 90 → Subsidy: ₱1,200 | Net: ₱4,600 × 0.95 = **₱3,990.00** Diet: High-Protein Growth	QUARANTINE
P-103	Barnaby • Rabbit • Age 4 Vax: Yes • Days: 5 VIP: No • Target Adopted: Yes	Base: ₱1,200 | Boarding (5×₱100): ₱500 Subsidy: ₱0.00 | Net: ₱1,700 × 0.95 = **₱1,615.00** Diet: Standard Balanced Organic	ADOPTED
P-104	Jasper • Dog • Age 10 Vax: Yes • SpecialNeeds: Yes Days: 45 • VIP: Yes (20% off)	Base: ₱3,500 | Boarding (45×₱250): ₱11,250 Risk: 55 → Subsidy: ₱600 | Net: ₱14,150 × 0.80 = **₱13,442.50** Diet: Senior Mobility & Renal	AVAILABLE

9. Grading & Verification Checklist

☐ **Modular Architecture:** Parsing, screening, fee calculation, and mutation are cleanly decomposed into static methods in `PetCare.Core` returning named C# tuples. No monolithic `Main` method.

☐ **Delimiter Token Extraction:** Uses `IndexOf`, `Substring`, `Remove`, and `TrimStart` to parse key-value tokens without regex or `.Split()`.

☐ **Defensive Input Pipelines:** Terminal prompts employ `do-while`/`while` loops with `int.TryParse` and `decimal.TryParse`; zero runtime crashes on bad input.

☐ **In-Place Array Mutation via For-Loops:** Directly mutates status values using indexed `for` loops (overcoming the `foreach` read-only constraint).

☐ **Reverse Traversal Mastery:** Successfully implements reverse-order record viewing using countdown indexing (`for (int i = count - 1; i >= 0; i--)`).

☐ **Safe Resource Disposal:** All file streams (`StreamReader` and `StreamWriter`) are strictly encapsulated inside `using` statements.

☐ **Zero Compilation Warnings:** Compiles under .NET 8 SDK with 0 errors and 0 warnings.